

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВВГУ»)

ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И АНАЛИЗА ДАННЫХ
КАФЕДРА ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И СИСТЕМ

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №5
по дисциплине
«Информатика и программирование»

Студент		
гр. БИН-25-3	_____	Т. Г. Пономаренко
Ассистент		
преподавателя	_____	М.В. Водяницкий

Задание

Выполнить задания и оформить отчет по стандартам ВВГУ.

Задание 1. Написать функцию, которая конвертирует время из одной величины в другую. На вход подаются:

- число (величина времени)
- исходная единица измерения
- единица измерения, в которую нужно перевести

Функция должна вернуть конвертированное значение.

Пример:

Введите время для конвертации: 4 h m

Результат: 240m

Задание 2. Пользователь делает вклад в банке в размере a рублей сроком на n лет. Процент по вкладу зависит от суммы и срока.

Зависимость от суммы:

- каждые 10 000 рублей увеличивают ставку на 0.3%
- суммарное увеличение не может превышать 5%
- минимальный вклад — 30 000 рублей

Зависимость от срока:

- первые 3 года — 3%
- от 4 до 6 лет — 5%
- более 6 лет — 2%

Необходимо написать функцию, которая рассчитывает **прибыль пользователя без учета первоначально вложенной суммы**. Используется сложный процент.

Пример:

Ввод: 30000 3

Прибыль: 3648.67 руб.

Задание 3. Написать функцию для вывода всех простых чисел в заданном диапазоне. Нужно учитывать некорректные данные (например, начало больше конца или диапазон без простых чисел).

На вход подаются два числа: начало и конец диапазона (включительно). На выходе — список всех простых чисел или сообщение об ошибке.

Пример:

Начало диапазона: 1

Конец диапазона: 10

2 3 5 7

Пример:

Начало диапазона: 0

Конец диапазона: 1

Error!

Задание 4. Реализовать функцию сложения двух матриц. При сложении двух матриц получается новая матрица того же размера, где каждый элемент — это сумма элементов с тем же индексом из двух исходных матриц.

Ограничения:

- складывать можно только матрицы одинакового размера
- размер матрицы должен быть строго больше 2 (например, 3×3, 4×4 и т.д.)
- при нарушении условий нужно вывести сообщение об ошибке

На вход подаются:

- 1) размер матрицы n (для квадратной матрицы $n \times n$)
- 2) элементы первой матрицы (по строкам, через пробел)
- 3) элементы второй матрицы (в таком же формате)

Результат — новая матрица (в том же формате), либо сообщение об ошибке.

Задание 5. Написать функцию, которая определяет, является ли строка палиндромом. Палиндром — это строка, которая читается одинаково слева направо и справа налево (без учета пробелов, регистра и знаков препинания).

На вход подается строка. На выходе:

- Да, если это палиндром
- Нет, если это не палиндром

Пример:

А роза упала на лапу Азора

Да

Пример:

Алфавитный порядок

Нет

Содержание

1	Выполнение работы	3
1.1	Задание 1	3
1.2	Задание 2	3
1.3	Задание 3	3
1.4	Задание 4	3
1.5	Задание 5	4
1.6	Задание 6	4
1.7	Задание 7	5
1.8	Задание 8	5
1.9	Задание 9	6
1.10	Задание 10	6

1 Выполнение работы

1.1 Задание 1

Программа ищет все числа 3 в списке и заменяет их на число 30. Она работает через цикл по индексам списка: на каждом шаге проверяется, равен ли текущий элемент числу 3. Если условие выполняется, значение по этому индексу меняется на 30. В результате получается модифицированный список, где все тройки заменены на новое значение. На рисунке 1 представлен код программы.

```
1 lst = [1,2,3,4,5,6,7,8,3,10]
2 for i in range(len(lst)):
3     if lst[i] == 3:
4         lst[i] = 30
5 print(lst)
```

Рисунок 1 – Листинг программы для задания 1

1.2 Задание 2

Программа изменяет исходный список, возводя каждый его элемент в квадрат. Она проходит по всем индексам списка с помощью цикла, вычисляет квадрат текущего элемента и записывает результат на его место. В итоге выводится новый список, состоящий из квадратов исходных чисел. На рисунке 2 представлен код программы.

```
1 lst = [1,2,3,4,5]
2 for i in range(len(lst)):
3     lst[i] = lst[i]**2
4 print(lst)
```

Рисунок 2 – Листинг программы для задания 2

1.3 Задание 3

Программа ищет в списке все вхождения числа 3 и заменяет их на 30. Цикл проходит по индексам списка, сравнивая каждый элемент с числом 3. Если совпадение найдено, значение по этому индексу заменяется на 30. На выходе получается модифицированный список, в котором все тройки заменены на заданное число. На рисунке 3 представлен код программы.

```
1 lst = [1,3,5,7,9]
2 a = max(lst)
3 print(a / len(lst))
```

Рисунок 3 – Листинг программы для задания 3

1.4 Задание 4

Данный фрагмент кода пытается отсортировать кортеж. Он помещает операцию сортировки в блок try, который преобразует кортеж в отсортированный список с помощью

функции `sorted`, а затем снова создаёт из него кортеж. Если в процессе возникает какое-либо исключение (например, если бы элементы кортежа были несравнимы), программа переходит в блок `except`, и переменной `result` просто присваивается исходный кортеж без изменений. В конце выводится полученный результат. На рисунке 4 представлен код решения.

```

1 t = (5, 2, 8, 1, 3)
2 try:
3     result = tuple(sorted(t))
4 except:
5     result = t
6 print(result)

```

Рисунок 4 – Листинг программы для задания 4

1.5 Задание 5

Код находит товары с минимальной и максимальной ценой в словаре, где ключи — названия продуктов, а значения — их цены. Для этого используются функции `min` и `max` с параметром `key`, который указывает, что сравнивать нужно по значениям словаря (ценам). В результате определяются ключи (названия товаров), соответствующие минимальной и максимальной цене, после чего выводятся оба значения вместе с ценами. На рисунке 5 представлен код программы.

```

1 products = {'яблоки': 100, 'бананы': 50, 'апельсины': 150, '
    молоко': 80, 'хлеб': 40}
2
3 min_p = min(products, key=products.get)
4 max_p = max(products, key=products.get)
5
6 print(f"Минимальная цена: {min_p} - {products[min_p]}")
7 print(f"Максимальная цена: {max_p} - {products[max_p]}")

```

Рисунок 5 – Листинг программы для задания 5

1.6 Задание 6

Программа создаёт словарь из списка разнотипных элементов, используя их строковые представления в качестве ключей. Для каждого элемента в исходном списке генерируется пара "ключ-значение" где ключом становится результат преобразования элемента в строку с помощью функции `str`, а значением — сам исходный элемент. Таким образом, в итоговом словаре ключи — это строки, а значения сохраняют оригинальные типы данных. На рисунке 6 представлен код программы.

```

1 elements = ["apple", 42, 3.14, True, "hello"]
2 result_dict = {str(element): element for element in elements}
3 print(result_dict)

```

Рисунок 6 – Листинг программы для задания 6

1.7 Задание 7

Эта программа создаёт англо-русский словарь, а затем инвертирует его, получая русско-английский. Для инверсии используется генератор словаря, который меняет местами ключи и значения исходного словаря. Пользователю предлагается ввести слово на русском языке, и если оно присутствует среди ключей нового словаря, выводится его английский перевод. В противном случае программа сообщает, что слово не найдено. На рисунке 7 представлен код программы.

```
1 eng_rus_dict = {  
2     "apple": "яблоко",  
3     "banana": "банан",  
4     "dog": "собака",  
5     "wear": "носить",  
6     "knee": "колени"  
7 }  
8  
9 rus_eng_dict = {value: key for key, value in eng_rus_dict.  
10     items()}  
11  
12 russian_word = input("Введите русское слово: ")  
13  
14 if russian_word in rus_eng_dict:  
15     print(f"Перевод: {rus_eng_dict[russian_word]}")  
16 else:  
17     print("Слово не найдено в словаре")
```

Рисунок 7 – Листинг программы для задания 7

1.8 Задание 8

Это реализация расширенной версии игры "Камень-ножницы-бумага" с добавлением ящерицы и Спока. Программа определяет победителя по заданным правилам, которые хранятся в словаре, где ключ — выбор игрока, а значение — список вариантов, которые этот выбор побеждает. После получения выбора пользователя и случайного выбора компьютера программа проверяет условия: ничья, победа пользователя (если выбор компьютера содержится в списке побеждаемых вариантов для выбора пользователя) или победа компьютера. На рисунке 8 представлен код программы.

```

1 import random
2
3 def rock_paper_scissors_lizard_spock():
4     choices = ["камень", "ножницы", "бумага", "ящерица", "спок"]
5     rules = {
6         "камень": ["ножницы", "ящерица"],
7         "ножницы": ["бумага", "ящерица"],
8         "бумага": ["камень", "спок"],
9         "ящерица": ["бумага", "спок"],
10        "спок": ["камень", "ножницы"]
11    }
12
13    print("Доступные варианты: камень, ножницы, бумага, ящерица, спок")
14    user_choice = input("Ваш выбор: ").lower()
15
16    comp_choice = random.choice(choices)
17    print(f"Компьютер выбрал: {comp_choice}")
18
19    if user_choice == comp_choice:
20        print("Ничья!")
21    elif comp_choice in rules[user_choice]:
22        print("Вы победили!")
23    else:
24        print("Компьютер победил!")
25
26 rock_paper_scissors_lizard_spock()

```

Рисунок 8 – Листинг программы для задания 8

1.9 Задание 9

Код группирует слова из списка фруктов по их первой букве, создавая словарь. Для каждого фрукта определяется его первая буква. Если этой буквы ещё нет среди ключей словаря, создаётся новый ключ с пустым списком в качестве значения. Затем фрукт добавляется в список, соответствующий своей начальной букве. В результате получается словарь, где ключи — это первые буквы, а значения — списки фруктов, которые с них начинаются. На рисунке 9 представлен код программы.

```

1 fruits = ["яблоко", "груша", "банан", "киви", "апельсин", "ананас"]
2 result = {}
3 for fruit in fruits:
4     first_letter = fruit[0]
5     if first_letter not in result:
6         result[first_letter] = []
7     result[first_letter].append(fruit)
8 print(result)

```

Рисунок 9 – Листинг программы для задания 9

1.10 Задание 10

Код вычисляет средний балл для каждого студента и определяет лучшего. На основе списка кортежей (имя, оценки) строится словарь, где ключ — имя студента, а значение — его средний балл. Для нахождения студента с максимальным средним используется

функция `max` с параметром `key`, которая указывает сравнивать не ключи словаря, а их значения. В результате выводится имя лучшего студента и его средний балл. На рисунке 10 представлен код программы.

```
1 students = [("Анна", [5, 4, 5]), ("Иван", [3, 4, 4]), ("
2     Мария", [5, 5, 5])]
3 student_grades = {}
4 for name, grades in students:
5     student_grades[name] = sum(grades) / len(grades)
6
7 best_student = max(student_grades, key=student_grades.get)
8 best_grade = student_grades[best_student]
9
10 print(f"\n{best_student} имеет наивысший средний балл: {
    best_grade:.1f}")
```

Рисунок 10 – Листинг программы для задания 10

Спасибо за внимание !